Test Case Specification Suite

SEOS — Software Engineering Operating System

Running Case Study: FoodDash

Course: UCS 503 — Software Engineering (BE Third Year, CSE, Group L2 [3P12])

Thapar Institute of Engineering & Technology, Patiala

Tanveer Chawla (1024160035) · Divyanshi Dhawan (1024160057) · Harshit Garg (1024160064)

Faculty Mentor: Er. Sandeep Kaur

60 test cases across 12 subsystems · Design date 02/11/2026

Test Case Index

TC #	Test Case Name	Subsystem	Runner
1.1	Generate Requirements from Product Idea	Requirements Engineering	Cypress Runner
1.2	Edit, Duplicate and Delete a Requirement	Requirements Engineering	Cypress Runner
1.3	Approve All Requirements Triggers Story Derivation	Requirements Engineering	Cypress Runner
1.4	Rejected Requirement Excluded from User Stories	Requirements Engineering	Vitest Runner
1.5	Generate with Empty Product Idea	Requirements Engineering	Cypress Runner
2.1	INVEST Stories with Gherkin Acceptance Criteria	User Stories & Acceptance Criteria	Cypress Runner
2.2	Story-to-Requirement Trace Badge	User Stories & Acceptance Criteria	Vitest Runner
3.1	Visual Model Downstream Staleness Flagging	Domain Modeling & Real-Time Sync	Vitest & Cypress Runner
3.2	Generated Model with PK/FK Badges and Cardinalities	Domain Modeling & Real-Time Sync	Cypress Runner
3.3	Canvas Drag, Zoom, Lock and Undo/Redo	Domain Modeling & Real-Time Sync	Cypress Runner
3.4	Entity Editor Field and Method Management	Domain Modeling & Real-Time Sync	Cypress Runner
3.5	Debounced Auto-Sync Status Transitions	Domain Modeling & Real-Time Sync	Cypress Runner
3.6	Offline Status and Local Data Retention	Domain Modeling & Real-Time Sync	Cypress Runner
4.1	Architecture Recommendation and Trade-offs	Architecture & Trade-offs	Cypress Runner
4.2	Literature-Grounded RAG Citations	Architecture & Trade-offs	Vitest Runner
5.1	Generate Valid OpenAPI 3.1 Contract	API Designer & OpenAPI Validation	Cypress Runner
5.2	Validator Flags Undeclared Path Parameter	API Designer & OpenAPI Validation	Vitest Runner
5.3	Validator Flags Invalid Root and Info Blocks	API Designer & OpenAPI Validation	Vitest Runner
5.4	Validator Warnings for Missing Responses and Schemas	API Designer & OpenAPI Validation	Vitest Runner
5.5	Sync Spec to Canvas (Reverse Parser)	API Designer & OpenAPI Validation	Vitest Runner
5.6	Endpoint Filtering and YAML Editor Tab Handling	API Designer & OpenAPI Validation	Cypress Runner
6.1	Generate SRS, SDD and README	Living Documentation	Cypress Runner
6.2	Documentation Staleness Banner and Regenerate	Living Documentation	Cypress Runner
6.3	Export Documentation as Markdown and PDF	Living Documentation	Cypress Runner
7.1	Full Six-Stage Pipeline Execution	Generation Pipeline & LLM Gateway	Vitest Runner
7.2	Resume Pipeline After LLM Failure	Generation Pipeline & LLM Gateway	Vitest Runner
7.3	Gemini Overload Retry with Exponential Backoff	Generation Pipeline & LLM Gateway	Vitest Runner
7.4	Dynamic Gemini Model Discovery and Ranking	Generation Pipeline & LLM Gateway	Vitest Runner
7.5	JSON Sanitizer and Provider Routing	Generation Pipeline & LLM Gateway	Vitest Runner
7.6	Transient State Excluded from Persistence	Generation Pipeline & LLM Gateway	Vitest Runner
8.1	Register and Login as Solo Developer	Authentication & RBAC	Pytest Runner
8.2	Student Registration with Valid Join Code	Authentication & RBAC	Pytest Runner
8.3	Student Registration with Invalid Join Code	Authentication & RBAC	Pytest Runner
8.4	Login Failure and Duplicate Email	Authentication & RBAC	Pytest Runner
8.5	JWT Content and Protected Endpoint Access	Authentication & RBAC	Pytest Runner
8.6	Role Enforcement on Mentor-Only Endpoints	Authentication & RBAC	Pytest Runner
8.7	One-Click Test Logins in Auth Modal	Authentication & RBAC	Cypress Runner
9.1	Mentor Creates Classroom with Unique Join Code	Classrooms & Mentor Roster	Pytest Runner
9.2	Student Joins Classroom by Code	Classrooms & Mentor Roster	Pytest Runner
9.3	Mentor Cohort Roster Shows Live Progress	Classrooms & Mentor Roster	Cypress Runner
9.4	Roster Access Denied for Non-Mentors	Classrooms & Mentor Roster	Cypress Runner

TC #	Test Case Name	Subsystem	Runner
10.1	Project CRUD Lifecycle	Project Persistence & Atomic Sync	Pytest Runner
10.2	Full Atomic Project Sync	Project Persistence & Atomic Sync	Pytest Runner
10.3	Atomic Rollback on Invalid Sync Payload	Project Persistence & Atomic Sync	Pytest Runner
10.4	Granular Artifact Endpoints	Project Persistence & Atomic Sync	Pytest Runner
10.5	Cascade Delete of Child Artifacts	Project Persistence & Atomic Sync	Pytest Runner
10.6	Project Visibility by Role	Project Persistence & Atomic Sync	Pytest Runner
10.7	Health Probe	Project Persistence & Atomic Sync	Pytest Runner
11.1	Upload and Chunk a Markdown Document	Knowledge Base & RAG	Pytest Runner
11.2	Upload a PDF Document	Knowledge Base & RAG	Pytest Runner
11.3	Reject Unsupported File Type	Knowledge Base & RAG	Pytest Runner
11.4	Search Literature by Keyword	Knowledge Base & RAG	Pytest Runner
11.5	List and Delete Uploaded Documents	Knowledge Base & RAG	Pytest Runner
11.6	Mentor Syllabus Available Class-Wide	Knowledge Base & RAG	Pytest Runner
12.1	Assistant Tool Calling: Add Requirement	Assistant & Workspace Shell	Cypress Runner
12.2	Assistant Citations Drawer	Assistant & Workspace Shell	Cypress Runner
12.3	Context Tab Document Ingestion	Assistant & Workspace Shell	Cypress Runner
12.4	Hash Routing for All Views	Assistant & Workspace Shell	Cypress Runner
12.5	Responsive Workspace Shell	Assistant & Workspace Shell	Cypress Runner
12.6	Learning Dashboard Progress and Concept Graph	Assistant & Workspace Shell	Cypress Runner

Test Case Specification (Doc: T_01)

Test Case #: 1.1
Test Case Name: Generate Requirements from Product Idea
System: SEOS (Software Engineering Operating System)
Subsystem: Requirements Engineering
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that a natural-language product idea produces 8–12 structured IEEE 830 requirements.

Pre-conditions

- A valid LLM provider API key is configured in Settings.
- A new, empty project is open; store.requirements is empty.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Navigate to Requirements Engineer (#/workspace/requirements)	Idea input box and preset list are displayed.		
2	Enter "Campus Carpooling System" and click Generate	Pipeline progress indicator runs; requirement cards appear.		
3	Count the generated cards	Between 8 and 12 cards are shown, including both Functional and Non-Functional types.		
4	Inspect ID, Priority and Status on each card	IDs are sequential from REQ-101; priorities are Critical/High/Medium/Low; status is Draft.		

Post-conditions

1. store.requirements contains 8–12 items, all with status Draft.
2. Requirement IDs are unique and follow the REQ-1xx pattern.

Test Case Specification (Doc: T_01)

Test Case #: 1.2
Test Case Name: Edit, Duplicate and Delete a Requirement
System: SEOS (Software Engineering Operating System)
Subsystem: Requirements Engineering
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify inline editing, duplication, priority change and deletion of requirement cards.

Pre-conditions

- Requirements have been generated for the project (test case 1.1).

Step	Action	Expected System Response	Pass / Fail	Comment
1	Edit the description of REQ-101 inline and confirm	Card shows the updated text.		
2	Change the priority of REQ-101 to Critical	Priority badge updates to Critical.		
3	Click Duplicate on REQ-101	A new card with identical content and a new unique ID is added.		
4	Click Delete on the duplicated card	Card is removed; remaining count returns to the original value.		

Post-conditions

1. store.requirements reflects the edit, priority change and deletion.
2. No duplicate requirement IDs exist.

Test Case Specification (Doc: T_01)

Test Case #: 1.3
Test Case Name: Approve All Requirements Triggers Story Derivation
System: SEOS (Software Engineering Operating System)
Subsystem: Requirements Engineering
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that Approve All moves Draft requirements to Approved and triggers user story derivation.

Pre-conditions

- A valid LLM provider API key is configured in Settings.
- Generated requirements exist, all with status Draft.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Click "Approve All"	All requirement cards change status from Draft to Approved.		
2	Observe the pipeline progress indicator	Story derivation starts automatically.		
3	Navigate to User Stories (#/workspace/user-stories)	User stories derived from the approved requirements are listed.		

Post-conditions

1. Every requirement in store.requirements has status Approved.
2. store.userStories is non-empty.

Test Case Specification (Doc: T_01)

Test Case #: 1.4
Test Case Name: Rejected Requirement Excluded from User Stories
System: SEOS (Software Engineering Operating System)
Subsystem: Requirements Engineering
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify that stories are derived only from Approved requirements.

Pre-conditions

- Requirements exist; REQ-102 is set to Rejected, all others are Approved.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Run generateStories() with the current store state	Story generation completes without error.		
2	Inspect reqId of every generated story	No story references REQ-102.		
3	Inspect stories for approved requirements	Each approved requirement is referenced by at least one story.		

Post-conditions

1. No user story in store.userStories has reqId = REQ-102.

Test Case Specification (Doc: T_01)

Test Case #: 1.5
Test Case Name: Generate with Empty Product Idea
System: SEOS (Software Engineering Operating System)
Subsystem: Requirements Engineering
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that generation cannot be started without a product concept.

Pre-conditions

- A new project is open; the idea input box is empty.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Click Generate with the idea box empty	No LLM request is sent; a validation message or disabled button prevents generation.		
2	Enter whitespace only and click Generate	Same behaviour as above; no cards are created.		

Post-conditions

1. store.requirements remains empty.
2. store.pipeline.running is false.

Test Case Specification (Doc: T_01)

Test Case #: 2.1
Test Case Name: INVEST Stories with Gherkin Acceptance Criteria
System: SEOS (Software Engineering Operating System)
Subsystem: User Stories & Acceptance Criteria
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify the structure of generated user stories and their Given/When/Then criteria.

Pre-conditions

- Approved requirements exist and stories have been generated.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Navigate to User Stories (#/workspace/user-stories)	Story cards are displayed.		
2	Read the statement of each story	Each follows "As a <role>, I want <action>, so that <benefit>".		
3	Count acceptance criteria per story	Each story has 3–4 criteria.		
4	Inspect the wording of the criteria	Each criterion is written as Given / When / Then.		

Post-conditions

1. Every story in store.userStories has role, action, benefit and 3–4 criteria.

Test Case Specification (Doc: T_01)

Test Case #: 2.2
Test Case Name: Story-to-Requirement Trace Badge
System: SEOS (Software Engineering Operating System)
Subsystem: User Stories & Acceptance Criteria
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify that each story is traceable to its parent requirement.

Pre-conditions

- Stories have been generated from approved requirements.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Open the User Stories view	A trace badge (e.g. US-101 → REQ-101) is shown on every story.		
2	Compare each badge with story.reqId in the store	Badge text matches the stored reqId.		
3	Look up each reqId in store.requirements	Every reqId resolves to an existing requirement.		

Post-conditions

1. No story has an orphan or empty reqId.

Test Case Specification (Doc: T_01)

Test Case #: 3.1

Test Case Name: Visual Model Downstream Staleness Flagging

System: SEOS (Software Engineering Operating System)

Subsystem: Domain Modeling & Real-Time Sync

Designed by: SEOS QA Team

Design Date: 02/11/2026

Executed by: Vitest & Cypress Runner

Execution Date: 02/11/2026

Short Description: Verify that modifying an entity field in the UML canvas flags downstream contracts (OpenAPI & Docs) as stale.

Pre-conditions

- Project "FoodDash" is open and synchronized across all stages.
- Initial stale state: { api: false, docs: false, architecture: false }.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Navigate to System Modeling (02/UML)	Interactive canvas renders existing entities (User, Order, Restaurant).		
2	Open Order entity editor, add field "discount code: String", and click Save	Canvas updates Order node; autosave fires toast [● SAVED].		
3	Inspect store state and downstream flags	Store sets staleArtifacts.api = true and staleArtifacts.docs = true.		
4	Navigate to API Designer (03/API)	Amber banner displayed: "Upstream model changes detected. API contract out of date."		
5	Click "Regenerate API Spec" on the warning banner	Spec regenerates with discount_code; banner disappears and flag resets to false.		

Post-conditions

1. Database contains updated Order entity schema with discount_code.
2. OpenAPI 3.1 contract and SRS documentation reflect the new field.

Test Case Specification (Doc: T_01)

Test Case #: 3.2
Test Case Name: Generated Model with PK/FK Badges and Cardinalities
System: SEOS (Software Engineering Operating System)
Subsystem: Domain Modeling & Real-Time Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that generated entities show PK/FK badges, data types and relationship cardinalities without overlap.

Pre-conditions

- A valid LLM provider API key is configured in Settings.
- Approved requirements and stories exist for FoodDash.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Run model generation and open System Modeling	Entity cards appear on the canvas.		
2	Inspect the key fields of each entity	Primary keys carry a yellow PK badge; foreign keys carry a cyan FK badge.		
3	Inspect the connecting edges	Cubic Bezier lines are labelled with cardinalities (1..1, 1..*, 0..*).		
4	Check node positions	Automatic layout places entities without overlapping.		

Post-conditions

1. Every entity in store.umlNodes has at least one field with pk = true.
2. Every FK field is linked by an edge in store.umlEdges.

Test Case Specification (Doc: T_01)

Test Case #: 3.3
Test Case Name: Canvas Drag, Zoom, Lock and Undo/Redo
System: SEOS (Software Engineering Operating System)
Subsystem: Domain Modeling & Real-Time Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify canvas interactions and the keyboard-supported history stack.

Pre-conditions

- A model with at least three entities is displayed on the canvas.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Drag the Order entity to a new position	Node moves and connected edges follow.		
2	Press Ctrl+Z	Node returns to its previous position.		
3	Press Ctrl+Y	Node moves back to the new position.		
4	Use zoom in, zoom out and reset view	Canvas scales and then returns to the default view.		
5	Enable lock mode and try to drag a node	Node does not move while lock mode is on.		

Post-conditions

1. Final node coordinates in store.umlNodes match the last unlocked position.

Test Case Specification (Doc: T_01)

Test Case #: 3.4
Test Case Name: Entity Editor Field and Method Management
System: SEOS (Software Engineering Operating System)
Subsystem: Domain Modeling & Real-Time Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that fields and methods can be added, changed and removed in the entity editor.

Pre-conditions

- System Modeling is open with the Restaurant entity present.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Open the Restaurant entity editor	Modal lists existing fields, types and methods.		
2	Add field "rating" of type DECIMAL	Field appears in the editor list.		
3	Toggle the FK flag on an existing field and change its type	Badge and type update in the editor.		
4	Add method "updateRating()" and remove an unused field	Method added; removed field disappears.		
5	Click Save	Canvas node shows the changes.		

Post-conditions

1. store.umlNodes reflects all field and method changes.
2. staleArtifacts.api and staleArtifacts.docs are true.

Test Case Specification (Doc: T_01)

Test Case #: 3.5
Test Case Name: Debounced Auto-Sync Status Transitions
System: SEOS (Software Engineering Operating System)
Subsystem: Domain Modeling & Real-Time Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify the 1.5 s debounce and the SYNCING / SAVED indicator in the top bar.

Pre-conditions

- User is logged in; a project is linked to a backend project ID.
- Backend running on port 8000 with seeded database.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Edit a requirement and watch the top bar	Store updates instantly; no network call yet.		
2	Type several further edits within 1.5 seconds	Debounce timer restarts with each keystroke; still no call.		
3	Stop typing and wait 1.5 seconds	Status shows [↑ SYNCING] and one PUT /projects/{id}/sync-full is sent.		
4	Wait for the response	Status changes to [● SAVED].		

Post-conditions

1. Exactly one sync request was issued for the burst of edits.
2. Backend data equals the latest client state.

Test Case Specification (Doc: T_01)

Test Case #: 3.6
Test Case Name: Offline Status and Local Data Retention
System: SEOS (Software Engineering Operating System)
Subsystem: Domain Modeling & Real-Time Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify behaviour when the backend becomes unreachable during editing.

Pre-conditions

- User is logged in with an active synced project.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Stop the backend (or block /api/v1) and edit an entity	UI updates locally without errors.		
2	Wait for the debounce to elapse	Top bar shows [OFFLINE].		
3	Reload the browser page	Edits are still present (restored from local storage).		
4	Restart the backend and make another edit	Status moves to [↑ SYNCING] then [● SAVED].		

Post-conditions

1. Backend project reflects all edits made while offline once sync succeeds.

Test Case Specification (Doc: T_01)

Test Case #: 4.1
Test Case Name: Architecture Recommendation and Trade-offs
System: SEOS (Software Engineering Operating System)
Subsystem: Architecture & Trade-offs
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify the structure of the generated architecture recommendation.

Pre-conditions

- A valid LLM provider API key is configured in Settings.
- Requirements and model exist for FoodDash.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Run architecture generation and open Architecture (#/workspace/architecture)	Recommendation page is displayed.		
2	Read the recommended pattern	One style is shown (Modular Monolith, Event-Driven Microservices or Layered Clean Architecture).		
3	Read the rationale	Rationale cites specific functional and non-functional requirement IDs.		
4	Inspect the components table	5–8 components with Name, Tech Stack and Purpose.		
5	Count the trade-offs	Exactly 3 trade-offs, each with an advantage and a cost.		

Post-conditions

1. store.architecture holds pattern, rationale, components and exactly 3 trade-offs.

Test Case Specification (Doc: T_01)

Test Case #: 4.2
Test Case Name: Literature-Grounded RAG Citations
System: SEOS (Software Engineering Operating System)
Subsystem: Architecture & Trade-offs
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify that architectural decisions carry citations from the curated literature base.

Pre-conditions

- An architecture recommendation has been generated.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Open the Grounded Literature References card	Citations list author, title and topic.		
2	Check each citation against knowledgeBase	Each belongs to one of the eight curated texts (e.g. Newman, Kleppmann, Fowler).		
3	Query the knowledge base with topic "modular monolith"	Fowler's Patterns of Enterprise Application Architecture is returned.		

Post-conditions

1. store.architecture.citations is non-empty and all entries resolve in the knowledge base.

Test Case Specification (Doc: T_01)

Test Case #: 5.1
Test Case Name: Generate Valid OpenAPI 3.1 Contract
System: SEOS (Software Engineering Operating System)
Subsystem: API Designer & OpenAPI Validation
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that the generated API contract is structurally valid and complete.

Pre-conditions

- A valid LLM provider API key is configured in Settings.
- A domain model exists for FoodDash.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Run API generation and open API Designer (#/workspace/api-designer)	YAML contract is displayed in the editor.		
2	Check the header badge	Badge reads Valid OpenAPI 3.1.0 ✓.		
3	Inspect the Endpoints tab	Operations are derived from entities and stories, with parameters and request bodies.		
4	Inspect responses of each operation	Responses 200/201, 400 and 404 are defined where applicable.		
5	Check the YAML length	Contract is under 300 lines.		

Post-conditions

1. store.openapiYaml parses without errors and contains components.schemas.

Test Case Specification (Doc: T_01)

Test Case #: 5.2
Test Case Name: Validator Flags Undeclared Path Parameter
System: SEOS (Software Engineering Operating System)
Subsystem: API Designer & OpenAPI Validation
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify that a path template parameter without a declaration produces an error with a line number.

Pre-conditions

- A valid contract containing /users/{userId} exists.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Remove the userId entry from the parameters list of /users/{userId}	Editor content changes.		
2	Run validateOpenApi() on the YAML	Result contains an Error naming userId.		
3	Open the Validation Report tab	Error is shown in red with the exact line number.		
4	Check the header badge	Badge shows an error counter instead of Valid.		

Post-conditions

1. Diagnostics list contains at least one error for the undeclared parameter.

Test Case Specification (Doc: T_01)

Test Case #: 5.3
Test Case Name: Validator Flags Invalid Root and Info Blocks
System: SEOS (Software Engineering Operating System)
Subsystem: API Designer & OpenAPI Validation
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify detection of a wrong openapi version, empty info title and paths without a leading slash.

Pre-conditions

- A valid contract is loaded in the editor.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Change root version to openapi: 3.0.0	Validator reports an error that 3.1.0 is required.		
2	Clear info.title	Validator reports a missing or empty title.		
3	Rename a path from /orders to orders	Validator reports that paths must begin with "/".		

Post-conditions

1. Diagnostics list shows three distinct errors, each with a line number.

Test Case Specification (Doc: T_01)

Test Case #: 5.4
Test Case Name: Validator Warnings for Missing Responses and Schemas
System: SEOS (Software Engineering Operating System)
Subsystem: API Designer & OpenAPI Validation
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify warning-level diagnostics for operations without responses and a missing components.schemas block.

Pre-conditions

- A valid contract is loaded.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Delete the responses block of one operation	Validator reports a warning for the operation.		
2	Delete components.schemas	Validator reports that components.schemas is missing.		
3	Open the Validation Report tab	Warnings appear in amber, separate from errors (red) and info (blue).		

Post-conditions

1. Diagnostics are categorized correctly as Error, Warning or Info.

Test Case Specification (Doc: T_01)

Test Case #: 5.5
Test Case Name: Sync Spec to Canvas (Reverse Parser)
System: SEOS (Software Engineering Operating System)
Subsystem: API Designer & OpenAPI Validation
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify that pasted OpenAPI YAML is converted into UML entities, types, keys, methods and edges.

Pre-conditions

- Valid OpenAPI YAML containing Order and Restaurant schemas, an order.restaurant_id property and POST /orders is available.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Paste the YAML into the editor and click "Sync Spec to Canvas"	Reverse parser runs and a statistics summary is displayed.		
2	Inspect field types of the created entities	string → VARCHAR, integer → INTEGER, date-time → TIMESTAMP, uuid → UUID.		
3	Inspect keys	id and *_id fields marked PK; restaurant_id marked FK.		
4	Inspect methods on Order	POST /orders maps to Order.create(); DELETE /orders/{id} maps to Order.delete().		
5	Open System Modeling	New entities and the inferred Order–Restaurant edge appear on the canvas.		

Post-conditions

1. store.umlNodes and store.umlEdges include the new entities and edges.
2. Statistics report entities created, updated and edges mapped.

Test Case Specification (Doc: T_01)

Test Case #: 5.6
Test Case Name: Endpoint Filtering and YAML Editor Tab Handling
System: SEOS (Software Engineering Operating System)
Subsystem: API Designer & OpenAPI Validation
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify method colour coding, endpoint filtering and 2-space indentation with the Tab key.

Pre-conditions

- A contract with GET, POST, PUT and DELETE operations is loaded.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Open the Endpoints tab	GET is green, POST cyan, PUT/PATCH amber, DELETE red.		
2	Apply a filter for POST	Only POST operations remain visible.		
3	Open the YAML editor, place the cursor on a line and press Tab	Two spaces are inserted and focus stays in the editor.		

Post-conditions

1. YAML content contains the two-space indentation and still validates.

Test Case Specification (Doc: T_01)

Test Case #: 6.1
Test Case Name: Generate SRS, SDD and README
System: SEOS (Software Engineering Operating System)
Subsystem: Living Documentation
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that the living documentation stage produces the three expected documents.

Pre-conditions

- A valid LLM provider API key is configured in Settings.
- All upstream stages (requirements to API) are complete.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Run documentation generation and open Documentation (#/workspace/documentation)	Document selector shows SRS, SDD and README.md.		
2	Open the SRS	IEEE 830 structure is rendered with headers, lists and tables.		
3	Open the SDD and README	Both render as GitHub-Flavored Markdown, including code blocks.		

Post-conditions

1. store.docs contains exactly three sections: SRS, SDD and README.

Test Case Specification (Doc: T_01)

Test Case #: 6.2
Test Case Name: Documentation Staleness Banner and Regenerate
System: SEOS (Software Engineering Operating System)
Subsystem: Living Documentation
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that a model edit marks documentation stale and that regeneration clears the flag.

Pre-conditions

- Documentation exists and staleArtifacts.docs is false.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Edit an entity in System Modeling and save	staleArtifacts.docs becomes true.		
2	Open Documentation	Amber staleness banner is displayed.		
3	Click "Regenerate Docs"	Documents regenerate and include the model change.		

Post-conditions

1. staleArtifacts.docs is false and the banner is no longer shown.

Test Case Specification (Doc: T_01)

Test Case #: 6.3
Test Case Name: Export Documentation as Markdown and PDF
System: SEOS (Software Engineering Operating System)
Subsystem: Living Documentation
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify the one-click export of documents.

Pre-conditions

- Documentation exists for the active project.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Click the Markdown export button in the top bar	A .md file is downloaded.		
2	Open the downloaded file	Content matches the on-screen documents.		
3	Click the PDF export button	A printable view or .pdf is produced with the formatted documents.		

Post-conditions

1. Exported content equals store.docs at the time of export.

Test Case Specification (Doc: T_01)

Test Case #: 7.1
Test Case Name: Full Six-Stage Pipeline Execution
System: SEOS (Software Engineering Operating System)
Subsystem: Generation Pipeline & LLM Gateway
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify that runPipeline() executes all stages in order and writes validated output to the store.

Pre-conditions

- LLM gateway is mocked to return valid JSON for every stage.
- Product idea is set.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Call runPipeline()	pipeline.running becomes true.		
2	Observe stage order	Order is requirements, stories, model, architecture, API, docs.		
3	Inspect the store after completion	Each stage output is written to its store field.		

Post-conditions

1. pipeline.running is false and all six artifacts are non-empty.

Test Case Specification (Doc: T_01)

Test Case #: 7.2
Test Case Name: Resume Pipeline After LLM Failure
System: SEOS (Software Engineering Operating System)
Subsystem: Generation Pipeline & LLM Gateway
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify that generation halts cleanly on failure and resumes from the first incomplete stage.

Pre-conditions

- LLM mock is set to fail at stage 4 (architecture).

Step	Action	Expected System Response	Pass / Fail	Comment
1	Call runPipeline()	Stages 1–3 complete; stage 4 fails and generation halts.		
2	Inspect the store	Requirements, stories and model are preserved; architecture is empty.		
3	Restore the mock and call resumePipeline()	Execution starts at stage 4.		
4	Count LLM calls for stages 1–3 after resume	Zero repeated calls.		

Post-conditions

1. All six artifacts are populated.
2. Stages 1–3 were each called exactly once.

Test Case Specification (Doc: T_01)

Test Case #: 7.3
Test Case Name: Gemini Overload Retry with Exponential Backoff
System: SEOS (Software Engineering Operating System)
Subsystem: Generation Pipeline & LLM Gateway
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify retry on HTTP 500/503 with delays of 2 s, 4 s and 8 s.

Pre-conditions

- Fake timers are enabled; the Gemini endpoint mock returns 503 three times, then 200.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Send a generation request	First attempt returns 503; a friendly status notice is set in llmNotice.		
2	Advance timers by 2 s, 4 s and 8 s in turn	Retries are issued at those delays.		
3	Allow the fourth attempt to complete	Request succeeds and the notice is cleared.		

Post-conditions

1. Exactly four requests were made.
2. The generated result is returned to the caller.

Test Case Specification (Doc: T_01)

Test Case #: 7.4
Test Case Name: Dynamic Gemini Model Discovery and Ranking
System: SEOS (Software Engineering Operating System)
Subsystem: Generation Pipeline & LLM Gateway
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify that the best available text model is selected without a hardcoded name.

Pre-conditions

- The model registry mock returns Flash, Pro, embedding and non-text models.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Call the model discovery routine	Registry is queried once.		
2	Inspect the filtered list	Non-text models are excluded.		
3	Apply rankGeminiModel to the candidates	A Flash variant is preferred for speed and rate limits.		

Post-conditions

1. store.activeModel holds the selected model name.

Test Case Specification (Doc: T_01)

Test Case #: 7.5
Test Case Name: JSON Sanitizer and Provider Routing
System: SEOS (Software Engineering Operating System)
Subsystem: Generation Pipeline & LLM Gateway
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify that parseJSON tolerates noisy responses and that Anthropic calls use the dev proxy.

Pre-conditions

- Test fixtures: a response wrapped in ```json fences, one with preamble text, one with trailing tokens.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Pass each fixture to parseJSON	A clean object is returned for each fixture.		
2	Pass completely invalid text	A controlled error is raised; no crash.		
3	Select Anthropic and send a request	Request is sent to /anthropic-proxy.		
4	Select OpenAI and send a request	Request is sent to api.openai.com with JSON schema enforcement.		

Post-conditions

1. No unhandled exceptions occurred.

Test Case Specification (Doc: T_01)

Test Case #: 7.6
Test Case Name: Transient State Excluded from Persistence
System: SEOS (Software Engineering Operating System)
Subsystem: Generation Pipeline & LLM Gateway
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Vitest Runner
Execution Date: Pending

Short Description: Verify that partialize stores project data but not transient execution flags.

Pre-conditions

- Store uses persist key seos-storage-v2.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Set pipeline.running = true, isThinking = true and llmNotice, and add a requirement	State is updated in memory.		
2	Read localStorage seos-storage-v2	Project data, apiKeys and theme are present; transient flags are absent.		
3	Rehydrate the store	Requirement, keys and theme are restored; running and isThinking are false.		

Post-conditions

1. Store is not locked in a running state after reload.

Test Case Specification (Doc: T_01)

Test Case #: 8.1
Test Case Name: Register and Login as Solo Developer
System: SEOS (Software Engineering Operating System)
Subsystem: Authentication & RBAC
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify code-free registration, login and session inspection for solo_dev.

Pre-conditions

- Backend running on port 8000 with seeded database.
- Email dev.test@example.com is not registered.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /auth/register with name, email, password and role solo_dev	Success response; user is created.		
2	POST /auth/login with the same credentials	Response contains an access token and profile data.		
3	GET /auth/me with the Bearer token	Returns the same email and role solo_dev.		

Post-conditions

1. users table holds one new row with a hashed (not plain-text) password.

Test Case Specification (Doc: T_01)

Test Case #: 8.2
Test Case Name: Student Registration with Valid Join Code
System: SEOS (Software Engineering Operating System)
Subsystem: Authentication & RBAC
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify that a student is enrolled automatically when registering with a valid join code.

Pre-conditions

- Backend running on port 8000 with seeded database.
- Classroom with code UCS503-FALL26 exists.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /auth/register with role student and join_code UCS503-FALL26	Success response.		
2	GET /auth/me with the new token	Profile lists the UCS503-FALL26 enrollment.		

Post-conditions

1. enrollments table contains a row linking the student and the classroom.

Test Case Specification (Doc: T_01)

Test Case #: 8.3
Test Case Name: Student Registration with Invalid Join Code
System: SEOS (Software Engineering Operating System)
Subsystem: Authentication & RBAC
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify that registration is refused when the join code does not exist.

Pre-conditions

- Backend running on port 8000 with seeded database.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /auth/register with role student and join code BAD-CODE	Request is rejected with a 4xx error and a clear message.		
2	POST /auth/login with the same email	Login fails because the account was not created.		

Post-conditions

1. No user and no enrollment row were created.

Test Case Specification (Doc: T_01)

Test Case #: 8.4
Test Case Name: Login Failure and Duplicate Email
System: SEOS (Software Engineering Operating System)
Subsystem: Authentication & RBAC
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify rejection of wrong passwords and of repeated email registration.

Pre-conditions

- Backend running on port 8000 with seeded database.
- student@thapar.edu is registered.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /auth/login with a wrong password	401 response; no token issued.		
2	POST /auth/login with an unknown email	401 response; no token issued.		
3	POST /auth/register again with student@thapar.edu	Request is rejected because email is unique.		

Post-conditions

1. users table still has a single row for student@thapar.edu.

Test Case Specification (Doc: T_01)

Test Case #: 8.5
Test Case Name: JWT Content and Protected Endpoint Access
System: SEOS (Software Engineering Operating System)
Subsystem: Authentication & RBAC
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify token claims and that protected endpoints reject missing or invalid tokens.

Pre-conditions

- Backend running on port 8000 with seeded database.
- A valid token for mentor@thapar.edu has been obtained.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Decode the access token payload	Claims sub (email), id and role are present.		
2	GET /auth/me without a token	401 Unauthorized.		
3	GET /auth/me with a tampered token	401 Unauthorized.		
4	GET /auth/me with the valid token	200 with the mentor profile.		

Post-conditions

1. Only the request with the valid token returned user data.

Test Case Specification (Doc: T_01)

Test Case #: 8.6
Test Case Name: Role Enforcement on Mentor-Only Endpoints
System: SEOS (Software Engineering Operating System)
Subsystem: Authentication & RBAC
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify that students and solo developers cannot use mentor-only operations.

Pre-conditions

- Backend running on port 8000 with seeded database.
- Tokens exist for a student and a mentor.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /classrooms with the student token	403 Forbidden.		
2	GET /classrooms/{id}/roster with the student token	403 Forbidden.		
3	POST /classrooms with the mentor token	Classroom is created.		

Post-conditions

1. Only the mentor request changed the database.

Test Case Specification (Doc: T_01)

Test Case #: 8.7
Test Case Name: One-Click Test Logins in Auth Modal
System: SEOS (Software Engineering Operating System)
Subsystem: Authentication & RBAC
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that the seeded Solo Dev, Student and Mentor logins work from the Auth modal.

Pre-conditions

- Frontend and backend are running; database is seeded.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Open the Auth modal and use 1-click login as Mentor	User is logged in; Cohort Roster is accessible.		
2	Log out and use 1-click login as Student	Student session starts; Cohort Roster is not accessible.		
3	Log out and use 1-click login as Solo Dev	Session starts with the FoodDash case study loaded.		

Post-conditions

1. store.currentUser matches the role of the last login.

Test Case Specification (Doc: T_01)

Test Case #: 9.1
Test Case Name: Mentor Creates Classroom with Unique Join Code
System: SEOS (Software Engineering Operating System)
Subsystem: Classrooms & Mentor Roster
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify classroom creation and join-code uniqueness.

Pre-conditions

- Backend running on port 8000 with seeded database.
- Logged in as mentor@thapar.edu.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /classrooms with name and description	Classroom is created and a join code is returned.		
2	Create a second classroom	A different join code is generated.		
3	GET /classrooms	Both classrooms are listed.		

Post-conditions

1. classrooms table has two rows with distinct code values and mentor_id set.

Test Case Specification (Doc: T_01)

Test Case #: 9.2
Test Case Name: Student Joins Classroom by Code
System: SEOS (Software Engineering Operating System)
Subsystem: Classrooms & Mentor Roster
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify enrollment of an existing student through POST /classrooms/join.

Pre-conditions

- Backend running on port 8000 with seeded database.
- A solo/student user is logged in and not yet enrolled; classroom UCS503-FALL26 exists.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /classrooms/join with code UCS503-FALL26	Enrollment succeeds.		
2	POST /classrooms/join with the same code again	Request does not create a duplicate enrollment.		
3	POST /classrooms/join with an unknown code	Request is rejected with an error.		

Post-conditions

1. enrollments contains exactly one row for the student and classroom.

Test Case Specification (Doc: T_01)

Test Case #: 9.3
Test Case Name: Mentor Cohort Roster Shows Live Progress
System: SEOS (Software Engineering Operating System)
Subsystem: Classrooms & Mentor Roster
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that student project changes appear on the mentor audit roster.

Pre-conditions

- Student is enrolled in UCS503-FALL26 and has a synced project.
- Mentor account is available.

Step	Action	Expected System Response	Pass / Fail	Comment
1	As student, approve requirements and wait for [● SAVED]	Project version increments in the backend.		
2	Log in as mentor and open Cohort Roster (#/workspace/mentor-roster)	Roster for UCS503-FALL26 is displayed.		
3	Inspect the student row	Name, email, enrollment date, project title, version and artifact badges are shown.		
4	Compare badges with the student's artifacts	Completion badges match the artifacts that exist.		

Post-conditions

1. Roster data equals the persisted state of the student's project.

Test Case Specification (Doc: T_01)

Test Case #: 9.4
Test Case Name: Roster Access Denied for Non-Mentors
System: SEOS (Software Engineering Operating System)
Subsystem: Classrooms & Mentor Roster
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that the roster view and endpoint are unavailable to student and solo_dev roles.

Pre-conditions

- Logged in as student@thapar.edu.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Check the navigation for Cohort Roster	Entry is hidden or inaccessible.		
2	Enter #/workspace/mentor-roster in the address bar	Roster data is not shown; user is redirected or told access is denied.		
3	Call GET /classrooms/{id}/roster directly	403 Forbidden.		

Post-conditions

1. No roster data was returned to the student.

Test Case Specification (Doc: T_01)

Test Case #: 10.1
Test Case Name: Project CRUD Lifecycle
System: SEOS (Software Engineering Operating System)
Subsystem: Project Persistence & Atomic Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify create, read, update and delete of a project.

Pre-conditions

- Backend running on port 8000 with seeded database.
- Logged in as solo_dev.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /projects with title and idea	Project is created with a UUID and status active.		
2	GET /projects/{id}	Full project model is returned.		
3	PUT /projects/{id} with a new title	Title is updated.		
4	DELETE /projects/{id}, then GET /projects/{id}	Delete succeeds; GET returns 404.		

Post-conditions

1. The project no longer exists in the database.

Test Case Specification (Doc: T_01)

Test Case #: 10.2
Test Case Name: Full Atomic Project Sync
System: SEOS (Software Engineering Operating System)
Subsystem: Project Persistence & Atomic Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify that sync-full persists all artifacts in a single transaction.

Pre-conditions

- Backend running on port 8000 with seeded database.
- A project exists; a payload with requirements, stories, UML data, architecture, OpenAPI YAML and docs is prepared.

Step	Action	Expected System Response	Pass / Fail	Comment
1	PUT /projects/{id}/sync-full with the payload	200 response.		
2	GET /projects/{id}	All six artifact groups are returned and match the payload.		
3	Re-send the same call with changed requirements	Previous requirements are replaced, not duplicated.		

Post-conditions

1. Project version is incremented.
2. Each artifact table holds exactly the latest data.

Test Case Specification (Doc: T_01)

Test Case #: 10.3
Test Case Name: Atomic Rollback on Invalid Sync Payload
System: SEOS (Software Engineering Operating System)
Subsystem: Project Persistence & Atomic Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify that a failing sync leaves no partial changes.

Pre-conditions

- Backend running on port 8000 with seeded database.
- A project with previously synced data exists.

Step	Action	Expected System Response	Pass / Fail	Comment
1	PUT sync-full with valid requirements but a malformed architecture section	Request is rejected with a validation error.		
2	GET /projects/{id}	Requirements still match the earlier sync.		

Post-conditions

1. No artifact table contains data from the rejected request.

Test Case Specification (Doc: T_01)

Test Case #: 10.4
Test Case Name: Granular Artifact Endpoints
System: SEOS (Software Engineering Operating System)
Subsystem: Project Persistence & Atomic Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify the individual artifact sync endpoints.

Pre-conditions

- Backend running on port 8000 with seeded database.
- A project exists.

Step	Action	Expected System Response	Pass / Fail	Comment
1	PUT /artifacts/requirements	Requirements are stored.		
2	PUT /artifacts/stories and /artifacts/diagram	Stories and UML node/edge data are stored.		
3	PUT /artifacts/architecture and /artifacts/api-spec	Architecture and OpenAPI YAML are stored.		
4	PUT /artifacts/docs, then GET /projects/{id}	All artifacts are returned together.		

Post-conditions

1. Each artifact table holds the data sent to its endpoint.

Test Case Specification (Doc: T_01)

Test Case #: 10.5
Test Case Name: Cascade Delete of Child Artifacts
System: SEOS (Software Engineering Operating System)
Subsystem: Project Persistence & Atomic Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify that deleting a project removes all dependent rows.

Pre-conditions

- Backend running on port 8000 with seeded database.
- A project with all artifact types has been synced.

Step	Action	Expected System Response	Pass / Fail	Comment
1	DELETE /projects/{id}	Success response.		
2	Query requirements, user_stories, diagrams, architecture, api_specs and documents for the project	All queries return zero rows.		

Post-conditions

1. No orphan rows remain for the deleted project.

Test Case Specification (Doc: T_01)

Test Case #: 10.6
Test Case Name: Project Visibility by Role
System: SEOS (Software Engineering Operating System)
Subsystem: Project Persistence & Atomic Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify that students see only their own projects while mentors can audit the cohort.

Pre-conditions

- Backend running on port 8000 with seeded database.
- Two students in one classroom each own a project.

Step	Action	Expected System Response	Pass / Fail	Comment
1	GET /projects as student A	Only student A's project is listed.		
2	GET /projects/{id} for student B's project as student A	Access is denied (403 or 404).		
3	GET /projects as the mentor	Cohort projects are listed for audit.		

Post-conditions

1. No data of student B was returned to student A.

Test Case Specification (Doc: T_01)

Test Case #: 10.7
Test Case Name: Health Probe
System: SEOS (Software Engineering Operating System)
Subsystem: Project Persistence & Atomic Sync
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify that the health endpoint reports service and database state.

Pre-conditions

- Backend running on port 8000 with seeded database.

Step	Action	Expected System Response	Pass / Fail	Comment
1	GET /api/v1/health	200 response reporting database connectivity as OK.		
2	Make the database unavailable and call the endpoint again	Response reports a database problem.		

Post-conditions

1. Health status reflects the actual database state.

Test Case Specification (Doc: T_01)

Test Case #: 11.1
Test Case Name: Upload and Chunk a Markdown Document
System: SEOS (Software Engineering Operating System)
Subsystem: Knowledge Base & RAG
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify upload, sliding-window chunking (600 characters, 100 overlap) and tagging.

Pre-conditions

- Backend running on port 8000 with seeded database.
- Logged in as a user; a 2,000-character notes.md file is available.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /knowledge/upload with notes.md	Upload succeeds.		
2	Inspect knowledge base rows for the file	Multiple chunks of at most 600 characters exist.		
3	Compare consecutive chunks	Neighbouring chunks overlap by about 100 characters, breaking at newline or full stop where possible.		
4	Check chunk metadata	Each chunk carries uploader ID, file name and classroom ID.		

Post-conditions

1. Chunks are immediately returned by search_knowledge.

Test Case Specification (Doc: T_01)

Test Case #: 11.2
Test Case Name: Upload a PDF Document
System: SEOS (Software Engineering Operating System)
Subsystem: Knowledge Base & RAG
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify text extraction from PDF files through pypdf.

Pre-conditions

- Backend running on port 8000 with seeded database.
- A text-based PDF with known phrases is available.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /knowledge/upload with the PDF	Upload succeeds.		
2	GET /knowledge/search?q=<known phrase>	A chunk containing the phrase is returned.		

Post-conditions

1. Chunks from every page of the PDF are stored.

Test Case Specification (Doc: T_01)

Test Case #: 11.3
Test Case Name: Reject Unsupported File Type
System: SEOS (Software Engineering Operating System)
Subsystem: Knowledge Base & RAG
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify that only .pdf, .md and .txt files are accepted.

Pre-conditions

- Backend running on port 8000 with seeded database.

Step	Action	Expected System Response	Pass / Fail	Comment
1	POST /knowledge/upload with a .docx file	Request is rejected with a format error.		
2	POST /knowledge/upload with a .exe file	Request is rejected.		

Post-conditions

1. No chunks were added to knowledge_base.

Test Case Specification (Doc: T_01)

Test Case #: 11.4
Test Case Name: Search Literature by Keyword
System: SEOS (Software Engineering Operating System)
Subsystem: Knowledge Base & RAG
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify search by keyword, book title and topic.

Pre-conditions

- Backend running on port 8000 with seeded database.
- Curated literature and one uploaded document are indexed.

Step	Action	Expected System Response	Pass / Fail	Comment
1	GET /knowledge/search?q=bounded context	Results include Newman's Building Microservices.		
2	GET /knowledge/search?q=Kleppmann	Results include Designing Data-Intensive Applications.		
3	GET /knowledge/search?q=zzzzqx	Empty result set; no error.		

Post-conditions

1. Each result includes source title and chapter or page.

Test Case Specification (Doc: T_01)

Test Case #: 11.5
Test Case Name: List and Delete Uploaded Documents
System: SEOS (Software Engineering Operating System)
Subsystem: Knowledge Base & RAG
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify document listing and that deletion prunes indexed chunks.

Pre-conditions

- Backend running on port 8000 with seeded database.
- Two documents were uploaded by the current user.

Step	Action	Expected System Response	Pass / Fail	Comment
1	GET /knowledge/my-documents	Both documents are listed.		
2	DELETE /knowledge/documents/{filename} for one document	Success response.		
3	GET /knowledge/my-documents and search for a phrase from the deleted file	Only the other document remains; search no longer returns chunks of the deleted file.		

Post-conditions

1. knowledge_base has no chunks for the deleted file name.

Test Case Specification (Doc: T_01)

Test Case #: 11.6
Test Case Name: Mentor Syllabus Available Class-Wide
System: SEOS (Software Engineering Operating System)
Subsystem: Knowledge Base & RAG
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Pytest Runner
Execution Date: Pending

Short Description: Verify that a mentor's uploaded syllabus is searchable by enrolled students.

Pre-conditions

- Backend running on port 8000 with seeded database.
- Mentor and enrolled student accounts exist.

Step	Action	Expected System Response	Pass / Fail	Comment
1	As mentor, upload syllabus.pdf	Upload succeeds and chunks are tagged with the classroom ID.		
2	As the enrolled student, search for a phrase from the syllabus	Matching chunks are returned.		
3	As an unenrolled user, search for the same phrase	No chunks from the syllabus are returned.		

Post-conditions

1. Syllabus chunks are visible only to the classroom and the mentor.

Test Case Specification (Doc: T_01)

Test Case #: 12.1
Test Case Name: Assistant Tool Calling: Add Requirement
System: SEOS (Software Engineering Operating System)
Subsystem: Assistant & Workspace Shell
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that the assistant can act on the project through tool calls.

Pre-conditions

- A valid LLM provider API key is configured in Settings.
- A project is open; Chat tab is active.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Type "Add a requirement for driver tracking" and send	isThinking is true while the assistant responds.		
2	Observe the response	Assistant calls add requirement and confirms the action.		
3	Open Requirements Engineer	A new requirement for driver tracking is present.		

Post-conditions

1. store.requirements includes the new requirement with a unique ID.

Test Case Specification (Doc: T_01)

Test Case #: 12.2
Test Case Name: Assistant Citations Drawer
System: SEOS (Software Engineering Operating System)
Subsystem: Assistant & Workspace Shell
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify grounded answers with an expandable source list.

Pre-conditions

- A valid LLM provider API key is configured in Settings.
- Chat tab is active.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Ask "Why choose a modular monolith?"	Answer is displayed.		
2	Click "Show sources"	Drawer lists literature chapters and pages (e.g. Fowler, Newman).		

Post-conditions

1. Every cited source exists in the knowledge base.

Test Case Specification (Doc: T_01)

Test Case #: 12.3
Test Case Name: Context Tab Document Ingestion
System: SEOS (Software Engineering Operating System)
Subsystem: Assistant & Workspace Shell
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify upload of course notes from the Context tab of the right sidebar.

Pre-conditions

- Logged in; backend running.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Open the Context tab	Active project artifacts are listed with inclusion toggles.		
2	Upload a .md file in the Document Ingestion Portal	File appears in the document list.		
3	Ask the assistant a question about the file's content	Answer cites the uploaded document.		

Post-conditions

1. Uploaded document is listed in GET /knowledge/my-documents.

Test Case Specification (Doc: T_01)

Test Case #: 12.4
Test Case Name: Hash Routing for All Views
System: SEOS (Software Engineering Operating System)
Subsystem: Assistant & Workspace Shell
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify that each workspace route renders its view.

Pre-conditions

- Application is open at #/.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Open #/workspace/overview, requirements and user-stories	Project Overview, Requirements Engineer and User Stories render.		
2	Open #/workspace/modeling, architecture and api-designer	System Modeling, Architecture and API Designer render.		
3	Open #/workspace/documentation and learning	Living Documentation and Learning Dashboard render.		
4	Open #/	Landing page renders.		

Post-conditions

1. No route produced a blank screen or console error.

Test Case Specification (Doc: T_01)

Test Case #: 12.5
Test Case Name: Responsive Workspace Shell
System: SEOS (Software Engineering Operating System)
Subsystem: Assistant & Workspace Shell
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify sidebar behaviour at different viewport widths.

Pre-conditions

- Workspace is open on a 1440 px wide viewport.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Drag the right sidebar edge	Width changes between 320 px and 600 px and does not go beyond.		
2	Resize the viewport below 1280 px	Left sidebar collapses to an icon-only rail.		
3	Resize the viewport below 1180 px	Assistant becomes a floating overlay and does not compress the canvas.		

Post-conditions

1. Workspace content remains usable at every size.

Test Case Specification (Doc: T_01)

Test Case #: 12.6
Test Case Name: Learning Dashboard Progress and Concept Graph
System: SEOS (Software Engineering Operating System)
Subsystem: Assistant & Workspace Shell
Designed by: SEOS QA Team
Design Date: 02/11/2026
Executed by: Cypress Runner
Execution Date: Pending

Short Description: Verify mastery metrics for the six engineering modules.

Pre-conditions

- A project with some completed artifacts is open.

Step	Action	Expected System Response	Pass / Fail	Comment
1	Open Learning Dashboard (#/workspace/learning)	Six modules are listed, each with an ASCII progress bar.		
2	Complete a new stage (e.g. generate documentation) and return	Matching module percentage increases.		
3	Inspect the Concept Graph	Mastered concepts (e.g. REST API, DDD) are highlighted.		

Post-conditions

1. Displayed percentages match the completion state of the project.